

```
"""
```

Raghuram Manchiganti

IBM SkillsBuild Data Analytics with AI Academic Internship

Project: Retail Sales Analysis and AI-Based Sales Prediction

Run:

```
pip install -r requirements.txt
```

```
python Raghuram_Manchiganti_Retail_Sales_Analysis_AI.py
```

```
"""
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.compose import ColumnTransformer
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.ensemble import RandomForestRegressor
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# -----
```

```
# 1. Setup
```

```
# -----
```

```
np.random.seed(42)
```

```
sns.set_theme(style="whitegrid")
```

```
print("=" * 65)
```

```
print("Raghuram Manchiganti - RETAIL SALES ANALYSIS WITH AI")
```

```
print("=" * 65)
```

```
# -----
```

```
# 2. Create Retail Dataset
```

```
# -----
```

```
n = 1500
```

```
dates = pd.date_range("2025-01-01", "2025-12-31", periods=n)
```

```
categories = np.random.choice(  
    ["Technology", "Furniture", "Office Supplies", "Beauty"],  
    size=n,  
    p=[0.28, 0.24, 0.28, 0.20]  
)
```

```
regions = np.random.choice(  
    ["North", "South", "East", "West"],  
    size=n  
)
```

```
segments = np.random.choice(  
    ["Consumer", "Corporate", "Home Office"],  
    size=n,  
    p=[0.55, 0.28, 0.17]  
)
```

```
base_price = {  
    "Technology": 220,  
    "Furniture": 180,  
    "Office Supplies": 70,  
    "Beauty": 90  
}
```

```

unit_price = np.array(
    [base_price[c] for c in categories]
) * np.random.uniform(0.65, 1.45, n)

quantity = np.random.randint(1, 8, n)
discount = np.round(np.random.uniform(0.00, 0.30, n), 2)

season_factor = (
    1
    + 0.12
    * np.sin((dates.dayofyear.to_numpy() / 365) * 2 * np.pi)
)

sales = (
    unit_price
    * quantity
    * (1 - discount)
    * season_factor
)

sales = sales * np.random.normal(1.0, 0.08, n)
sales = np.maximum(sales, 10)

cost_rate = {
    "Technology": 0.68,
    "Furniture": 0.62,
    "Office Supplies": 0.55,
    "Beauty": 0.58
}

cost = sales * np.array(
    [cost_rate[c] for c in categories]
)

```

)

profit = sales - cost - (sales * discount * 0.12)

```
df = pd.DataFrame({
    "Date": dates,
    "Category": categories,
    "Region": regions,
    "Customer_Segment": segments,
    "Quantity": quantity,
    "Discount": discount,
    "Unit_Price": np.round(unit_price, 2),
    "Sales": np.round(sales, 2),
    "Profit": np.round(profit, 2)
})
```

df["Month"] = df["Date"].dt.to_period("M").astype(str)

print("\nDataset created successfully.")

print("Rows:", len(df))

print("Columns:", len(df.columns))

3. Data Validation

print("\n" + "-" * 65)

print("DATA VALIDATION")

print("-" * 65)

print("\nMissing values:")

print(df.isnull().sum())

```
print("\nDataset summary:")
```

```
print(df.describe())
```

```
# -----
```

```
# 4. Exploratory Data Analysis
```

```
# -----
```

```
# Monthly Sales
```

```
monthly_sales = (  
    df.groupby("Month", as_index=False)["Sales"]  
        .sum()  
)
```

```
plt.figure(figsize=(12, 5))
```

```
plt.plot(  
    monthly_sales["Month"],  
    monthly_sales["Sales"],  
    marker="o"  
)
```

```
plt.title("Monthly Sales Trend")
```

```
plt.xlabel("Month")
```

```
plt.ylabel("Total Sales")
```

```
plt.xticks(rotation=45)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# Sales by Category
```

```
category_sales = (  
    df.groupby("Category")["Sales"]  
        .sum()  
        .sort_values(ascending=False)
```

```
)

plt.figure(figsize=(9, 5))
category_sales.plot(kind="bar")
plt.title("Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.xticks(rotation=20)
plt.tight_layout()
plt.show()
```

```
# Profit by Region
region_profit = (
    df.groupby("Region")["Profit"]
    .sum()
    .sort_values(ascending=False)
)
```

```
plt.figure(figsize=(8, 5))
region_profit.plot(kind="bar")
plt.title("Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()
```

```
# Customer Segment Analysis
segment_summary = df.groupby(
    "Customer_Segment"
).agg(
    Total_Sales=("Sales", "sum"),
```

```
Total_Profit=("Profit", "sum"),
Avg_Order_Value=("Sales", "mean"),
Orders=("Sales", "count")
).sort_values(
    "Total_Sales",
    ascending=False
)

print("\nCustomer Segment Analysis:")
print(segment_summary)
```

```
# Correlation Matrix
```

```
numeric_cols = [
    "Quantity",
    "Discount",
    "Unit_Price",
    "Sales",
    "Profit"
]
```

```
correlation = df[numeric_cols].corr()
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f"
)
```

```
plt.title("Correlation Matrix")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

5. Business Insights

```
# -----
```

```
total_sales = df["Sales"].sum()
```

```
total_profit = df["Profit"].sum()
```

```
average_order = df["Sales"].mean()
```

```
profit_margin = (  
    total_profit / total_sales  
) * 100
```

```
top_category = category_sales.index[0]
```

```
top_region = region_profit.index[0]
```

```
top_segment = segment_summary.index[0]
```

```
print("\n" + "=" * 65)
```

```
print("BUSINESS INSIGHTS")
```

```
print("=" * 65)
```

```
print(f"Total Sales          : ${total_sales:,.2f}")
```

```
print(f"Total Profit         : ${total_profit:,.2f}")
```

```
print(f"Average Order Value   : ${average_order:,.2f}")
```

```
print(f"Overall Profit Margin   : {profit_margin:.2f}%")
```

```
print(f"Top Sales Category      : {top_category}")
```

```
print(f"Top Profit Region       : {top_region}")
```

```
print(f"Top Customer Segment    : {top_segment}")
```

```
# -----
```

6. AI / Machine Learning Model

```
# -----
```

```
print("\n" + "=" * 65)
```

```
print("AI / MACHINE LEARNING - SALES PREDICTION")
```

```
print("=" * 65)
```

```
features = [  
    "Category",  
    "Region",  
    "Customer_Segment",  
    "Quantity",  
    "Discount",  
    "Unit_Price"  
]
```

```
target = "Sales"
```

```
X = df[features]
```

```
y = df[target]
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42  
)
```

```
categorical_features = [  
    "Category",  
    "Region",  
    "Customer_Segment"  
]
```

```
numeric_features = [  
    "Quantity",  
    "Discount",
```

```
    "Unit_Price"
]

preprocessor = ColumnTransformer(
    transformers=[
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore"),
            categorical_features
        ),
        (
            "num",
            "passthrough",
            numeric_features
        )
    ]
)
```

```
model = RandomForestRegressor(
    n_estimators=250,
    random_state=42,
    max_depth=12,
    min_samples_leaf=2
)
```

```
pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", model)
])
```

```
pipeline.fit(X_train, y_train)
```

```
predictions = pipeline.predict(X_test)
```

```
# -----  
# 7. Model Evaluation  
# -----  
mae = mean_absolute_error(  
    y_test,  
    predictions  
)  
  
rmse = np.sqrt(  
    mean_squared_error(  
        y_test,  
        predictions  
    )  
)  
  
r2 = r2_score(  
    y_test,  
    predictions  
)  
  
print("\nModel Evaluation:")  
print(f"MAE : ${mae:,.2f}")  
print(f"RMSE : ${rmse:,.2f}")  
print(f"R2 : {r2:.3f}")  
  
# -----  
# 8. Actual vs Predicted Sales  
# -----  
plt.figure(figsize=(7, 6))
```

```
plt.scatter(  
    y_test,  
    predictions,  
    alpha=0.65  
)
```

```
min_value = min(  
    y_test.min(),  
    predictions.min()  
)
```

```
max_value = max(  
    y_test.max(),  
    predictions.max()  
)
```

```
plt.plot(  
    [min_value, max_value],  
    [min_value, max_value],  
    linestyle="--"  
)
```

```
plt.title("Actual vs Predicted Sales")  
plt.xlabel("Actual Sales")  
plt.ylabel("Predicted Sales")  
plt.tight_layout()  
plt.show()
```

```
# -----
```

```
# 9. Feature Importance
```

```
# -----
```

```
feature_names = (
```

```
pipeline
.named_steps["preprocessor"]
.get_feature_names_out()
)

importances = (
    pipeline
    .named_steps["model"]
    .feature_importances_
)

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(
    "Importance",
    ascending=False
).head(12)

print("\nTop Features:")
print(importance_df)

plt.figure(figsize=(10, 6))

plt.barh(
    importance_df["Feature"][:-1],
    importance_df["Importance"][:-1]
)

plt.title(
    "Top Features Influencing Sales Prediction"
)
```

```
plt.xlabel("Importance")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# -----
```

```
# 10. Example AI Prediction
```

```
# -----
```

```
new_transaction = pd.DataFrame({  
    "Category": ["Technology"],  
    "Region": ["South"],  
    "Customer_Segment": ["Consumer"],  
    "Quantity": [4],  
    "Discount": [0.10],  
    "Unit_Price": [250]  
})
```

```
predicted_sales = pipeline.predict(  
    new_transaction  
)[0]
```

```
print("\n" + "=" * 65)  
print("EXAMPLE AI PREDICTION")  
print("=" * 65)
```

```
print(  
    f"Predicted Sales for the example transaction: "  
    f"${predicted_sales:,.2f}"  
)
```

```
# -----
```

```
# 11. Final Summary
```

```
# -----
```

```
print("\n" + "=" * 65)
```

```
print("PROJECT COMPLETED SUCCESSFULLY")
```

```
print("=" * 65)
```

```
print("""
```

This project performed:

1. Retail data generation
2. Data validation
3. Exploratory Data Analysis
4. Sales and profit analysis
5. Data visualization
6. Business KPI calculation
7. Random Forest AI/ML model training
8. Model evaluation
9. Sales prediction
10. Feature importance analysis

Project: Retail Sales Analysis and AI-Based Sales Prediction

Author: Raghuram Manchiganti

```
""")
```